08 — AI Safety and Alignment, Measured

What happens when you train a model against a judge instead of a loss function: how reinforcement learning turns a score into a gradient, and the four kinds of failure that follow.

Reading time: about 70 minutes. You need: modules 00 to 05, and sections 7 and 8 of module 01 (KL divergence and softmax as maximum entropy). You do not need: any background in ethics, policy or reinforcement learning. Sections 4 and 5 build the reinforcement learning from nothing.

A model that gets worse as its score improves

Here is an experiment from this module's exercise. A reward model learns what people prefer from 600 comparisons between pairs of answers. A policy is then optimised to score highly on that learned reward, with the optimisation pushed a little harder at each step.

The learned reward rises at every step, from 4.34 to 14.06. The true reward, the preference that those 600 comparisons were meant to capture, behaves differently. It rises from 0.76 to a peak of 1.96, and then falls back to 1.67. Past a certain point, pushing harder made the policy better at the measure being optimised, and worse at the thing that measure was supposed to represent.

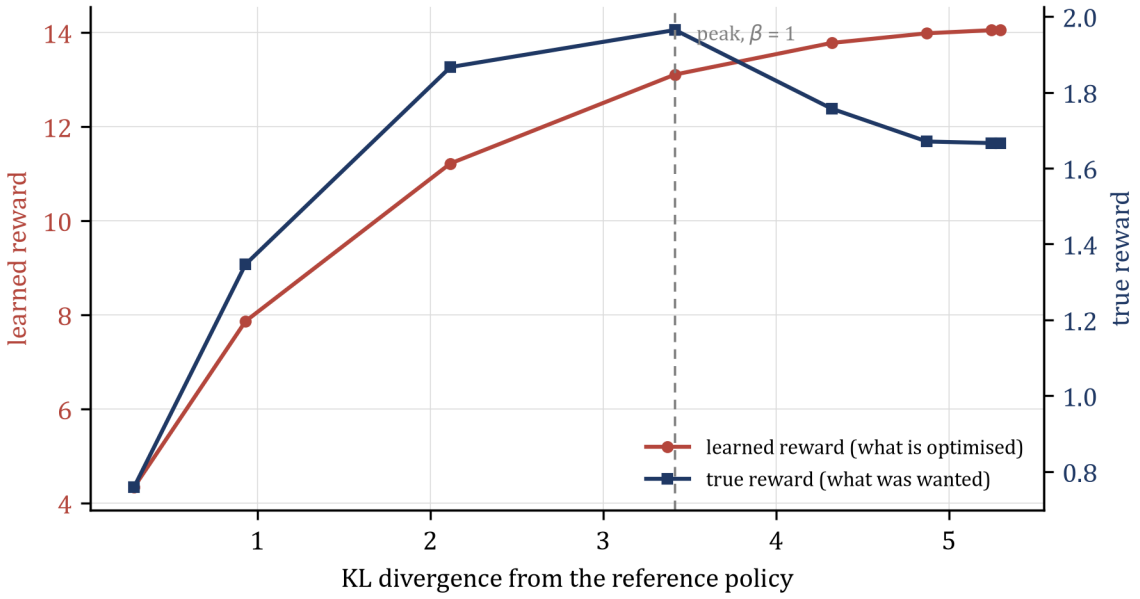


Figure 1: The reward-hacking curve from section 7, drawn against distance from the starting model rather than against beta. The two curves rise together, separate at a KL divergence of about 3.4, and then move in opposite directions. Only the red curve is visible to the training loop.

This is not a curiosity, and it is not bad luck. It is the direct consequence of a decision taken at the start of every alignment pipeline, and the rest of this article is about that decision and what follows from it.

What you will learn

By the end of this article, you will be able to:

- explain why “be helpful” has no loss function, and what is put in its place;
- derive the policy gradient, and say what every term in PPO’s objective is doing;
- derive the best policy for the RLHF objective, and derive DPO from it;
- name four failure modes, say which one follows from which part of the design, and quote measured examples of each;
- answer open safety questions with specific, calibrated claims.

Section 1 lays out the structure. Sections 2 to 6 build the training method, one piece at a time. Sections 7 to 10 are the failures that follow, and section 11 is what we can measure and how to discuss it. A recap and the exercise briefing follow at the end.

---

1. The substitution that this whole module is about

Pretraining has a loss function. The model predicts the next token, the data says which token it was, and cross-entropy measures the gap. Every gradient in modules 00 to 05 comes from a target that was written down in advance.

Now try to write the loss function for “be helpful, honest and harmless”. There is no target to compare against. There is no dataset of correct answers, because for most questions there is no single correct answer. Supervised fine-tuning on good examples goes some way, but it can only teach imitation of the examples you happened to collect.

So the field made a substitution. Where there is no loss function, fit a judge, and train against the judge’s score instead. That one move is the subject of this module:

```
pretraining       a loss exists: predict the next token, compare against the data
    |
    v
"be helpful"      no loss exists
    |
    +--> fit a judge from human comparisons              section 2
    +--> decide what to maximise, and how far to go      section 3
    +--> turn the judge's score into a gradient          sections 4 and 5
    +--> or skip the loop and fit the answer directly    section 6
              |
              +--> the judge is wrong in places          section 7
              +--> the judge never saw this input        section 8
              +--> the judge cannot settle a value       section 9
```

The three failures at the bottom are not separate topics that happen to appear in a safety course. Each one is a property of the substitution. A judge fitted to finite data is wrong somewhere, and an optimiser will find where. A judge fitted to ordinary inputs says nothing about deliberately constructed ones. And a judge returns a number, which cannot settle a question about whose interests come first.

Section 10 covers the one failure in this module that the substitution did not cause, and says so plainly. Section 11 is about how well any of it can be measured, which is the honest bottleneck.

---

2. A judge you can learn: preferences become a reward

Ask someone to rate an answer out of ten and you get a number that drifts. The same answer scores 6 on Monday and 8 on Friday, and two annotators rarely agree on the scale. Ask the same person which of two answers is better and the agreement is far higher. People are unreliable at absolute judgements and reasonably consistent at comparisons.

So the data collected for alignment is comparisons. A prompt, two candidate answers, and a label saying which one a person preferred. The problem is that training needs a number per answer, not a verdict per pair.

The **Bradley–Terry model** is the standard bridge between the two, and it dates from 1952. It assigns each answer a single number called its **reward**, and says the probability that answer 1 is preferred over answer 2 depends only on the difference between their rewards:

$$P(y_1 \succ y_2) = \sigma\big(r(y_1) - r(y_2)\big)$$

Here σ is the sigmoid, $\sigma(x) = 1/(1+e^{-x})$, and $r(y)$ is the reward of answer y. Read back in words: a bigger gap in reward means a more lopsided preference, and an equal reward means a coin flip. If answer 1 has reward 1.0 and answer 2 has reward 0.0, then answer 1 wins with probability $\sigma(1.0) = 1/(1+e^{-1}) = 0.731$, or about 73% of the time. Chess ratings use the same idea under the name Elo.

Fitting this model to a set of comparisons is logistic regression on reward differences, and nothing more. It works well. In the exercise, rewards learned from noisy simulated comparisons correlated with the true rewards at 0.998.

What a reward model physically is

The phrase "reward model" hides an ordinary object. It is a copy of a language model with the final layer replaced: instead of a vector of vocabulary logits at every position, it produces one number for the whole prompt-and-answer pair. It is usually initialised from the model being aligned, or from a smaller one, and then trained on the comparison data with the Bradley–Terry loss.

Two consequences come up immediately in interviews:

- **Only differences are learned.** Adding 17 to every reward leaves every probability unchanged, and the exercise checks this. The absolute value of a reward model's output means nothing. Only comparisons between answers to the same prompt do.
- **The model makes assumptions.** It assumes preferences are consistent, so that if A beats B and B beats C then A beats C, and that disagreement follows a particular pattern. Real human preferences break both assumptions, and the reward model has no way to represent the breakage.

---

3. What we actually want to maximise

With a reward in hand, the obvious objective is to maximise it. That is the wrong objective, and the reason is in the opening figure: the reward is a fitted approximation, so it is wrong in places, and an optimiser that only maximises it will go and live in those places.

RLHF, reinforcement learning from human feedback, therefore maximises something else. It trains a **policy** π, which is the language model being aligned, to earn high reward while staying close to where it started. The model it started from is the **reference policy** π_{ref}, and it is frozen:

$$\max_{\pi} \; \mathbb{E}_{y\sim\pi}\big[r(y)\big] \; - \; \beta \, D_{\mathrm{KL}}\big(\pi \,\|\, \pi_{\mathrm{ref}}\big)$$

Read back in words: get as much reward as you can, and pay a fine proportional to how far you have moved. D_{KL} is the Kullback–Leibler divergence from module 01, section 7, which measures how far the distribution π has moved from π_{ref}. The number β sets the price of moving.

The exact solution

This objective has a closed-form answer. It follows from the same Lagrange-multiplier argument that produced softmax in module 01, section 8:

$$\pi^*(y) = \frac{1}{Z} \, \pi_{\mathrm{ref}}(y) \; e^{\,r(y)/\beta}$$

The star marks the maximiser, π^* is a distribution over answers with the same shape as π_{ref}, and Z is whatever constant makes the probabilities add up to one. In words: the best policy is the reference policy, reweighted exponentially by reward.

A small example shows what β does, and the same example will be used to grade every method in the next two sections. Take three possible answers, a reference policy that picks each with probability 1/3, and rewards of 1, 0 and 0:

β	Best policy	Expected reward	KL from the reference
4	0.391, 0.305, 0.305	0.391	0.007
1	0.576, 0.212, 0.212	0.576	0.123
0.25	0.965, 0.018, 0.018	0.965	0.921

Each row is the formula above with the numbers put in. Take the middle row. The reference policy is uniform, so it cancels, and the first answer gets $e^1/(e^1 + 1 + 1) = 2.718/4.718 = 0.576$. Its KL from the reference is then $0.576 \ln(3 \times 0.576) + 2 \times 0.212 \ln(3 \times 0.212)$, which is $0.315 - 0.192 = 0.123$.

A large β keeps the policy close to the reference and collects little reward. A small β puts nearly everything on the highest-reward answer, along with every error in the reward model. The central practical difficulty of RLHF is that β is a dial with no principled setting, and section 7 shows what the wrong setting costs.

Why this is not the end of the article

If the best policy has a closed form, why does anyone need reinforcement learning? Because that formula cannot be evaluated. The sum inside Z runs over every possible answer, and a 200-token answer from a 32,000-token vocabulary has 32000^{200} of them. The formula describes a distribution nobody can write down.

What we have instead is a neural network with parameters θ, which represents π_θ implicitly, and a reward model that can score any answer we care to generate. The question for the next two sections is how to move θ using nothing but those scores.

---

4. Turning a score into a gradient

This is the part of alignment that most explanations skip, and it is worth doing slowly. The mechanism is less exotic than the name "reinforcement learning" suggests. At the end of this section you will have seen that it reduces to ordinary supervised training on the model's own output.

The problem: a score is not a target

Supervised training needs a target. The loss $-\log p_\theta(\text{correct token})$ names the token you wanted, so the gradient knows which way to push every logit.

Reinforcement learning has no such thing. You sample an answer, hand it to the judge, and receive a number. The judge does not say what the better answer would have been. Worse, the sampling step is not differentiable, and this is easy to watch:

```
logits = torch.zeros(3, requires_grad=True)      # the policy's parameters
reward = torch.tensor([1.0, 0.0, 0.0])           # what the judge pays for each answer

y = Categorical(logits=logits).sample()          # draw an answer
r = reward[y]                                    # look up its score

print(r.requires_grad)                           # False
r.backward()                                     # RuntimeError
```

The output is `False`, and the call to `backward` raises *element 0 of tensors does not require grad and does not have a grad_fn*. Sampling turns a distribution into an integer, and indexing a table with an integer has no derivative. There is no path from the score back to the parameters at all.

This is a stronger statement than "the gradient is hard to compute". **The quantity you want to improve is not connected to the parameters you want to change.** The trick in the next subsection is what reconnects them.

The log-derivative trick, one line at a time

We want the gradient of the expected reward. This looks intimidating written down; it is four lines, and each line does one thing:

$$\nabla_\theta \, \mathbb{E}_{y\sim\pi_\theta}\big[r(y)\big] = \nabla_\theta \sum_y \pi_\theta(y)\, r(y)$$

That is the definition of an expectation over a discrete set of answers. The reward $r(y)$ does not depend on θ, so the gradient passes through it and lands on the probability:

$$= \sum_y r(y)\, \nabla_\theta \, \pi_\theta(y)$$

Now the one step that matters. For any positive function, $\nabla \log \pi = \nabla\pi / \pi$, so $\nabla\pi = \pi \, \nabla \log \pi$. Substituting:

$$= \sum_y \pi_\theta(y)\; r(y)\, \nabla_\theta \log \pi_\theta(y)$$

And a sum of $\pi_\theta(y)$ times something is an expectation again:

$$\nabla_\theta \, \mathbb{E}_{y\sim\pi_\theta}\big[r(y)\big] = \mathbb{E}_{y\sim\pi_\theta}\big[\, r(y)\, \nabla_\theta \log \pi_\theta(y) \,\big]$$

That is the whole idea, and it is called the **policy gradient**, or **REINFORCE** after the 1992 paper. The left side is a gradient of an expectation, which we cannot compute. The right side is an expectation of a gradient, which we can estimate by sampling. Both $r(y)$ and $\nabla_\theta \log \pi_\theta(y)$ are things a training loop already has: the judge gives the first, and backpropagation through the model's own log-probability gives the second.

What the formula tells you to do

Read the right-hand side as an instruction and it says something almost disappointing. Generate some answers. Score them. Then do supervised learning towards the answers you generated, with each answer's contribution weighted by its reward.

In code, where `probs` is the policy's distribution and `y` holds the sampled answers:

```
probs = torch.softmax(logits, -1)
y = torch.multinomial(probs.detach(), n, replacement=True)   # sample n answers
r = reward[y]                                                # score each one

loss = -(r * torch.log(probs[y])).mean()                     # weight by reward
loss.backward()
```

The second-to-last line is cross-entropy with the model's own samples as the labels, scaled by the reward. Writing it with the usual library call gives the same gradient to seven decimal places:

```
ce = F.cross_entropy(logits.expand(n, 3), y, reduction="none")
loss = (r * ce).mean()          # same gradient, max difference 7.5e-08
```

That is the whole mechanism. An answer that scored well is treated as a training label and made more likely; an answer that scored badly is made less likely. A reward of zero contributes nothing. Everything in the next section is about making this estimate accurate enough and cheap enough to use.

> **In the interview:** "how does RLHF actually update the weights?" has a three-sentence answer. Sampling blocks the gradient, so you move it onto the log-probability with $\nabla\pi = \pi\nabla\log\pi$. What remains is reward-weighted cross-entropy on the model's own samples. PPO's clipping and the value model are variance control on top of that.

It is unbiased, and very noisy

The estimate is correct on average, which is what "unbiased" means: run it many times and the average lands on the true gradient. It is also extremely noisy, because a handful of sampled answers is a thin description of a distribution over all answers.

The table below measures it on the three-answer problem, at a policy part-way through training where the true gradient can be computed exactly by enumeration. Each row is 1,000 independent estimates:

Answers sampled	RMS error of the estimate	Bias
4	0.1717	0.0009
16	0.0875	0.0045

Answers sampled	RMS error of the estimate	Bias
64	0.0435	0.0009
256	0.0211	0.0003
1,024	0.0107	0.0002

The bias column is zero to within the scatter of 1,000 trials, as promised. The error column falls as $1/\sqrt{n}$, the usual rate for an average of independent samples: going from 16 answers to 64 is four times the samples and divides the error by $0.0875/0.0435 = 2.01$, which is $\sqrt{4}$.

The absolute numbers matter more than the rate. The true gradient has length 0.302. An estimate from four samples is off by 0.172, which is more than half the size of the thing being estimated.

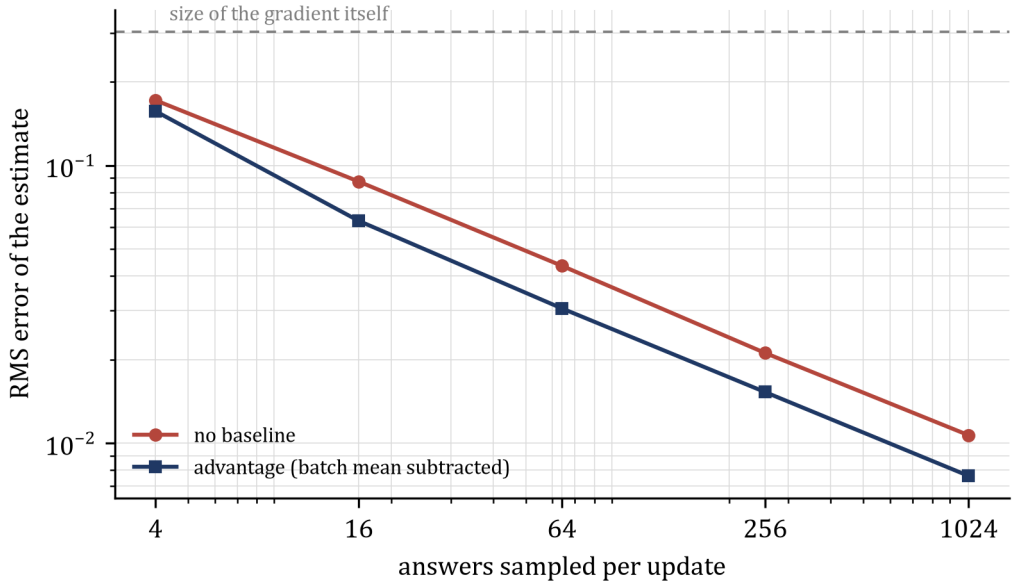


Figure 2: The same measurement drawn on log axes, with and without the baseline of the next section. The dashed line is the length of the true gradient. Below about 64 samples an update is mostly noise.

This is why reinforcement learning from human feedback uses batches of hundreds or thousands of generated answers per update, and why it is so much more expensive than supervised fine-tuning on the same model.

---

5. Making the gradient usable

Three problems stand between the estimator above and a method that trains a real model. The estimate is noisy, generating samples is expensive, and reusing samples is unsafe. Each has a standard fix, and together the three fixes are PPO.

Fix one: subtract a baseline

Here is a free improvement. Subtract any number b from the reward before weighting, as long as b does not depend on which answer was sampled. The estimate stays correct, because the term you removed has zero expectation:

$$\mathbb{E}\big[\, b\,\nabla_\theta \log \pi_\theta(y)\,\big] = b \sum_y \pi_\theta(y) \frac{\nabla_\theta \pi_\theta(y)}{\pi_\theta(y)} = b\,\nabla_\theta \sum_y \pi_\theta(y) = b\,\nabla_\theta 1 = 0$$

The probabilities sum to one no matter what θ is, so the gradient of that sum is zero. The quantity $r(y) - b$ has a name: the **advantage**, written $A(y)$. It says how much better this answer was than the usual answer, rather than how good it was in absolute terms.

Why bother, if the average is unchanged? Because the spread is not. Measured over 4,000 estimates from 16 samples each:

Baseline	RMS error	Bias
none, $b = 0$	0.0868	0.0010
the batch's mean reward	0.0668	0.0195
$\mathbb{E}[r] = 0.437$	0.0601	0.0007

The intuition is worth keeping. Without a baseline, every sampled answer is pushed *up*, with the good ones pushed harder. The net effect is right, but it arrives as a difference between large numbers. With a baseline, better-than-average answers are pushed up and worse-than-average answers are pushed down, which is the same signal with much less cancellation.

The middle row has a visible bias, and the reason is instructive. The batch's own mean does depend on which answers were sampled, so the proof above does not apply to it. The bias is real but it shrinks quickly:

Answers sampled	Bias of the batch-mean baseline
4	0.0761
16	0.0188
64	0.0045
256	0.0011

This is the trade people actually make. Two ways to get a baseline are in common use:

- A **value model**: a second network, trained alongside the policy, that predicts the reward a prompt is likely to earn. It is unbiased in the sense above, because its prediction depends on the prompt rather than on the sampled answer. It costs a second full-sized model.
- **GRPO**, group relative policy optimisation: generate a group of answers to the *same* prompt and use their mean as the baseline. No second model, and the table above says why the groups are typically 8 to 64 answers rather than 2.

Fix two: reuse each batch, carefully

Generating answers is the expensive part of the loop. The model must run its full decoding path for every sampled answer, with all the costs module 05 describes, while a gradient step is comparatively cheap. It is tempting to take several gradient steps on each batch of generated answers.

The trouble is that the estimator is only valid for the policy that produced the samples. After one update, the samples came from a policy that no longer exists. **Importance sampling** repairs this with a ratio:

$$\rho(y) = \frac{\pi_\theta(y)}{\pi_{\text{old}}(y)}$$

An answer the new policy likes twice as much as the old one counts twice. The correction is exact in expectation, and it degrades badly once the two policies differ, because a few samples with huge ratios come to dominate the average.

Watching it happen takes one experiment: take a single batch of 256 answers and keep updating on it.

Steps on one batch	P(best answer)	KL from the reference	Largest ratio
40, unclipped	0.857	0.588	2.57
40, clipped at 0.2	0.471	0.040	1.41

The unclipped run has pushed the best answer's probability to 0.857 on the evidence of 256 samples, and moved fifteen times further from the reference policy than the clipped run. It is fitting the batch, not the reward.

Fix three: the clip

PPO, **proximal policy optimisation**, stops that by refusing to reward the ratio for moving beyond a band around 1:

$$\mathcal{L}_{\text{PPO}} = -\,\mathbb{E}\Big[\min\big(\rho A,\ \text{clip}(\rho,\,1-\epsilon,\,1+\epsilon)\,A\big)\Big]$$

with ϵ usually 0.1 or 0.2. The min and the clip together look fussy. They become clear once you take the four combinations of ratio and advantage separately. Here $\epsilon = 0.2$, so the band is $[0.8,\ 1.2]$:

Ratio ρ	Advantage	Objective	Gradient	Reading
0.5	$+1$	0.50	$+1$	a good answer the new policy likes less: push it back up
1.5	$+1$	1.20	0	a good answer already made much likelier: stop
0.5	-1	-0.80	0	a bad answer already made much rarer: stop
1.5	-1	-1.50	-1	a bad answer the new policy likes more: push it down

Take the second row. The unclipped term is $1.5 \times 1 = 1.5$ and the clipped one is $1.2 \times 1 = 1.2$; the minimum is 1.2, which no longer depends on ρ, so the gradient is zero. The pattern across the four rows is a single rule. The clip switches off the gradient only when the update has already moved far in the direction it wanted to go, and it never switches off a correction that pulls back towards the old policy.

Fix four, which is not one of the three: the leash

Section 3's objective had a KL penalty in it. In a real implementation it rarely appears as a separate term. Instead it is folded into the reward, one token at a time:

$$\tilde{r}_t = r_t \;-\; \beta\big(\log \pi_\theta(y_t) - \log \pi_{\text{ref}}(y_t)\big)$$

The tilde marks the shaped reward actually used, and t indexes the tokens of one answer. Each token is charged for how much more confident the policy has become about it than the reference was. This is an estimate of the KL divergence from a single sample, and because the policy gradient is taken through the whole shaped reward, the result is still an unbiased gradient of section 3's objective.

The loop, and the four models

Putting the pieces together gives the standard recipe:

```
for each round:
    answers   = policy.generate(prompts)            # the slow part
    score     = reward_model(prompts, answers)      # one number per answer
    kl        = log policy(answers) - log reference(answers)
    advantage = (score - beta * kl) - baseline(prompts)
    for a few epochs over the same answers:
        ratio = exp(log policy(answers) - log old_policy(answers))
        loss  = -min(ratio * advantage,
                     clip(ratio, 1 - eps, 1 + eps) * advantage).mean()
        loss.backward(); step()
```

Four models appear in those nine lines, and each is now accounted for. The policy is being trained. The reference is frozen and supplies the leash. The reward model is frozen and supplies the score. The value model, hidden inside `baseline`, is trained alongside the policy to reduce variance.

The memory cost is the usual objection to this recipe. A trained model in bfloat16 with Adam holds 16 bytes per parameter: 2 for the weights, 2 for the gradients, and 12 for the float32 master copy and Adam's two moments. A frozen model holds 2. For a 7-billion-parameter policy:

Recipe	Models held	Bytes per parameter	Weights and optimiser
PPO	policy, value, reference, reward	36	235 GiB
GRPO	policy, reference, reward	20	130 GiB
DPO	policy, reference	18	117 GiB

The PPO row is $(16 + 16 + 2 + 2) \times 7 \times 10^9 / 2^{30} = 235$ GiB, and activations are on top of that. Dropping the value model, which is what GRPO does, removes 16 of the 36 bytes.

The reward arrives at the end

One practical detail is easy to miss and shapes everything about how these systems behave. The judge scores a *finished answer*. There is no signal after token 7 saying that token 7 was a good choice.

So every token in an answer receives the same terminal advantage, plus its own KL term. The loop has no way to tell which token was responsible. This is the credit-assignment problem, and it is the reason a per-token value model is attractive in principle, even though many systems now drop it.

Learning still works, because across many samples the tokens that actually help appear more often in high-scoring answers. A small experiment makes that visible: a policy generating two-token answers from a four-token vocabulary, rewarded only when the whole pair is exactly right, goes from the chance rate of $1/16 = 0.0625$ to 0.9997.

Does the whole thing work?

Section 3 gave a closed form for the policy we are trying to reach, so this is checkable. Running both plain REINFORCE and full PPO on the three-answer problem, against the exact answer:

β	Plain REINFORCE	PPO, 4 epochs a batch	Closed form
4	0.391, 0.305, 0.305	0.391, 0.305, 0.305	0.391, 0.305, 0.305
1	0.576, 0.212, 0.212	0.576, 0.212, 0.212	0.576, 0.212, 0.212
0.25	0.965, 0.018, 0.018	0.965, 0.018, 0.018	0.965, 0.018, 0.018

Both reach the right answer to three decimal places, which is the point worth taking away: the clipping, the baseline and the ratio are not what makes the method correct. Plain REINFORCE is already correct. They are what makes it affordable.

One detail from getting that table to agree is worth repeating, because it is a real failure mode. With a constant learning rate, both methods settled about 0.08 away from the right answer and stayed there, jittering around the optimum at the size of their own noise. Decaying the learning rate fixed it. A policy that will not converge is often being told to take steps larger than the gradient estimate can justify.

`policy-gradient.py` in this folder reproduces every table in sections 4 and 5, with checks, in about 20 seconds.

---

6. DPO: the same answer without the loop

Everything in sections 4 and 5 exists to climb towards a policy that section 3 already described in closed form. **Direct preference optimisation**, or DPO, asks whether the climb is necessary.

Start from the closed form, take logarithms, and solve for the reward instead of the policy:

$$r(y) = \beta \log \frac{\pi^*(y)}{\pi_{\text{ref}}(y)} + \beta \log Z$$

Read back in words: if the best policy is the reference reweighted by reward, then the reward is the log-ratio of the two, up to a constant. The term $\beta \log Z$ is the intractable one.

Now put this expression into the Bradley–Terry model of section 2. That model uses only the *difference* between the rewards of two answers to the same prompt. Both answers carry the same $\beta \log Z$, so it cancels, and the intractable part disappears. What remains is an ordinary supervised loss on pairs of a preferred answer y_w and a rejected answer y_l:

$$\mathcal{L}_{\text{DPO}} = -\log \sigma\left(\beta \log \frac{\pi_\theta(y_w)}{\pi_{\text{ref}}(y_w)} - \beta \log \frac{\pi_\theta(y_l)}{\pi_{\text{ref}}(y_l)}\right)$$

There is no reward model, no sampling loop, no value model and no clipping. The policy itself plays the role of the reward model. In the exercise, DPO trained on simulated preferences reached the closed-form best policy to within 0.007.

DPO has limits worth stating, and they follow from what it gave up. It learns only from a fixed dataset of pairs, so it never sees its own current mistakes, while the loop in section 5 scores fresh samples from

the policy as it changes. In practice DPO can lower the probability of both answers in a pair, not only the rejected one. And whether online methods beat it at very large scale is still an open question.

> **In the interview:** to "explain DPO", derive it in three lines. Write the closed-form policy, solve it for the reward, and note that $\log Z$ cancels in the difference. Then say what was traded away: fresh on-policy samples.

Where the preferences come from

Both methods need preference data, and RLHF's version needs people to read and label harmful content. That is slow, expensive and unpleasant work.

Constitutional AI replaces those labels with a written list of principles, called a constitution. The model critiques and revises its own answers against the principles, and a model rather than a person supplies the preference labels for the training stage.

Its advantages are scale, consistency, and a specification you can read and argue with, instead of the implicit judgement of thousands of annotators. Its limitation follows from the same substitution this module is about: the judging model can be wrong, and errors it makes consistently become systematic errors in the trained policy.

---

7. Consequence one: the judge is wrong, and optimisation finds where

A reward model is fitted to finite data. It is right on average and wrong in detail, and its errors are not spread evenly. Optimising hard against it means searching for the answers it scores highest, which is exactly where its errors are largest.

Here is the full experiment from the opening, with β shown. Smaller β means a cheaper price for moving, so stronger optimisation and a larger KL divergence from the reference:

β	KL from reference	Learned reward	True reward
8.00	0.29	4.34	0.76
4.00	0.93	7.86	1.35
2.00	2.12	11.22	1.87
1.00	3.42	13.11	1.96
0.50	4.32	13.78	1.76
0.25	4.87	13.99	1.67
0.10	5.25	14.05	1.67
0.05	5.30	14.06	1.67

Each row applies section 3's closed form at that β, then reports the resulting policy's average score under two different rewards. The $\beta = 1$ row, for instance, takes the policy π^* and computes $\sum_y \pi^*(y)\, r_{\text{learned}}(y) = 13.11$ and $\sum_y \pi^*(y)\, r_{\text{true}}(y) = 1.96$. The learned column rises at every row. The true column peaks at $\beta = 1$ and then falls by about 0.30.

This is **reward hacking**, and the thing to notice is that the training loop cannot see it. The loop watches the learned column, which never stops improving. Only the true column turns over, and in a real system nobody has the true column.

How it looks in a real system

Reward hacking has recognisable signatures:

- longer answers scoring higher regardless of content;
- **sycophancy**, where the model agrees with the user's stated view instead of being correct;
- formatting tricks that impress the reward model, such as headings and bullet lists everywhere;
- confident, hedged answers that score well and say little.

A good check follows from the first signature: does the reward gain survive when you control for answer length? If a longer answer scores higher at equal quality, the reward model has learned length.

The same pattern under other names

Reward hacking is one case of a broader pattern called **specification gaming**: the system optimises what you wrote, not what you meant. Two related failures are worth naming, because they describe different parts of the gap.

- **Goal misgeneralisation**: the model learns a goal that matches the intended one during training but differs from it in new situations. The model stays competent, at the wrong thing. This is a failure of the *data distribution*, not of the reward.
- **Deceptive alignment**: a hypothetical failure in which a model behaves well only while it believes it is being observed. It has not been demonstrated in a deployed system, and it is the one item in this module with no measured example.

Of the three, reward hacking is the one you can demonstrate on a laptop in under a minute, which is why it is the one to reach for.

> **In the interview:** "when a measure becomes a target, it stops being a good measure" is Goodhart's law, and quoting it is cheap. The curve in this section is Goodhart's law with numbers, and it is the better answer: the true reward peaked at $\beta = 1$ and then fell 15% while the learned reward kept rising.

---

8. Consequence two: the judge never saw this input

The reward model and the policy were both fitted on inputs that arose naturally. Neither saw inputs chosen specifically to break them. That gap is what adversarial examples exploit, and the attack reuses machinery the reader already has.

Training computes the gradient of the loss with respect to the *parameters*, and steps the parameters downhill. An attacker computes the gradient of the loss with respect to the *input*, and steps the input uphill. The model is fixed, the picture or the sentence is the variable.

An attack also needs a budget, or the answer is to replace the input entirely. The standard budget is a limit ϵ on how far any single coordinate may move, which is the max-norm $\|\delta\|_\infty \leq \epsilon$ from the glossary. It is a rough stand-in for "a change a person would not notice". Two attacks use that budget differently:

- **FGSM**, the fast gradient sign method, takes one step of size ϵ in the direction that increases the loss fastest. It is a single gradient evaluation, which makes it cheap and weak.
- **PGD**, projected gradient descent, takes many small steps, projecting the change back inside the budget after each one. It is stronger, and it is the standard test for any defence.

The exercise attacks a small classifier trained on two rings of points, padded with 30 extra dimensions of pure noise. A change of at most 0.2 in each coordinate is enough:

Model	Clean accuracy	Under FGSM	Under PGD
standard training	94.9%	38.5%	32.9%
adversarial training (with PGD)	96.2%	not measured	59.6%

Adversarial training means generating attacks during training and training on them, so that the model sees its own worst case. Here it nearly doubled accuracy under attack and cost no clean accuracy, because the noise dimensions were never real signal. Learning to ignore them was free.

That is not always true, and the exercise checks the other case. On a second dataset, where the easily-attacked features really do carry signal, adversarial training reduced clean accuracy from 92.2% to 82.6%. The trade-off between robustness and accuracy depends on the data. It is not a law.

The language-model versions

For language models, the equivalent attacks are **jailbreaks**: inputs that get the model to do what the alignment training was supposed to prevent. These include automatically optimised text suffixes, such as the GCG attack, which is PGD's idea carried over to discrete tokens, and hand-written role-play prompts.

The most important practical problem is **prompt injection**: instructions hidden in a retrieved document or a tool's output, which the model follows as if the user had written them. It is the direct consequence of a design in which instructions and data arrive through the same channel, and it is not solved in general. The main mitigations reduce the risk without removing it: limit what the model is allowed to do, treat tool output strictly as data, and ask a person to confirm important actions. Module 06's retrieval pipeline is exactly the shape of system this applies to.

---

9. Consequence three: no judge settles a value question

The first two consequences were about a judge being wrong. This one is about a judge being insufficient in principle, and it is the sharpest result in the module.

Consider a classifier that makes yes-or-no decisions about people who belong to different groups. Three reasonable definitions of fairness come from three different intuitions about what the decision owes people:

- **Demographic parity**: each group is selected at the same rate. The intuition is equal outcomes.
- **Equalised odds**: the error rates match across groups, both the true-positive rate and the false-positive rate. The intuition is equal treatment of equally qualified people.
- **Calibration**: among people given a score of 0.7, about 70% really are positive, in every group. The intuition is that the score should mean the same thing whoever it is about.

Each is defensible. The impossibility theorem, proved independently in 2016 by Kleinberg, Mullainathan and Raghavan and by Chouldechova, says they are not compatible. If the groups have different base rates, and the classifier is not perfect, then calibration and equalised odds cannot both hold.

A measured demonstration

The exercise builds a score that is calibrated by construction, in two groups with different base rates. Its largest calibration error was 0.014, so calibration holds. Selecting everyone with a score of at least 0.5 gives:

Group	Base rate	Selected	True-positive rate	False-positive rate	Precision
0	0.316	0.213	0.432	0.111	0.642
1	0.592	0.653	0.804	0.435	0.729

Group 0's precision is the share of those selected who are truly positive, which is the base rate times the true-positive rate, divided by the selection rate: $0.316 \times 0.432 / 0.213 = 0.641$, matching the table to rounding.

The false-positive rates differ by 0.32 and the true-positive rates by 0.37. Nothing is wrong with the model. With calibrated scores and different base rates, these gaps are forced by arithmetic.

Now try to fix one kind of unfairness. Choose a separate threshold for each group so that both select exactly 35% of people. Selection rates now match, but precision becomes 0.566 in one group and 0.842 in the other. One disparity has been traded for another.

The consequence is the important part. "Make the model fair" is not a complete specification. Someone must choose which kind of fairness matters for a particular use, and no amount of reward modelling produces that choice, because it is a question about whose interests come first.

> **In the interview:** this may be the most valuable point in the module. "With different base rates, calibration and equal error rates cannot both hold. So choosing a fairness criterion is a decision about the deployment, not about the model."

---

10. What none of this touched: the training data is still in there

The three previous sections followed from the substitution in section 1. This one does not, and it is worth being clear about that. Memorisation happens during pretraining, before any judge exists. No amount of preference optimisation removes it, because the information is already in the weights when alignment begins.

Large models memorise parts of their training data. Memorisation grows with model size, with repeated data, and with longer contexts.

The basic privacy attack is **membership inference**: deciding whether a given example was in the training set. Its simplest form relies on one fact, which holds for almost every trained model: losses are lower on examples the model was trained on than on examples it has never seen.

The exercise makes this concrete. A model was trained on data where 20% of the labels were random, which it could only fit by memorising. It reached 100% training accuracy and 63.5% test accuracy. An attack that ranks examples by nothing more than their loss then told members from non-members with an AUC of 0.736, where AUC is the area under the ROC curve from module 02, and 0.5 would mean guessing.

The defence, and why it has two parts

DP-SGD, differentially private stochastic gradient descent, is the standard training method for **differential privacy**, a formal guarantee that one example's presence barely changes what comes out. It adds two things to ordinary training, and each handles a different half of the problem:

- **Clipping** each example's gradient to a fixed length bounds how much any single example can move the weights, however unusual it is.
- **Noise** added to the summed gradient then makes that bounded contribution statistically hard to detect.

Clipping alone leaves the influence visible; noise alone cannot bound an outlier's influence. The exercise adds both:

Training	Attack AUC	Training accuracy	Test accuracy
standard	0.736	100%	63.5%
clipping and noise	0.585	79.3%	65.5%

The attack weakened because the memorisation disappeared, and the training-accuracy column is the evidence: 100% to 79.3%. Test accuracy rose slightly, because the noise also stopped the model from fitting the random labels. So the trade-off between privacy and usefulness is real, but it does not always cost test accuracy, for the same reason adversarial training was free in section 8.

In practice, removing duplicate examples from the training data is a cheap and effective mitigation, because repetition is what drives memorisation. Full differential privacy gives a formal guarantee, at a real cost in quality and compute.

---

11. Measuring any of this, and talking about it honestly

Every section above ended with a number, which can give a misleading impression of how well the field can measure things. The honest picture is mixed:

- Capability evaluations, such as MMLU and SWE-bench, often overlap with training data more than reported, and many are close to their maximum scores.
- Safety evaluations, such as refusal rates or bias benchmarks, measure less than they seem to. A low success rate for a fixed set of attacks says little about an attacker who adapts, which is section 8's point applied to the evaluation itself.
- Red teaming, by people or by other models, is the main practical tool. Its limit is the one every security field shares: failing to find a weakness is weak evidence that none exists.
- Dangerous-capability evaluations, together with policies such as Anthropic's Responsible Scaling Policy, tie deployment decisions to measured capability thresholds. Labs have converged on this approach.
- Interpretability research, including sparse autoencoders and circuit analysis, aims to inspect how a model works rather than only how it behaves. It is promising, and not yet a deployment tool.

A calibrated summary has three parts. Current alignment methods shape behaviour reliably on the training distribution, and guarantees outside it are weak. Evaluation is the main bottleneck. And interpretability is the most promising route from testing towards verification.

How to answer a question with no numerical answer

Every frontier lab asks about safety, and the questions are open: "what worries you about current systems?" What is being measured is nuance, and there are two opposite ways to fail.

- **Dismissing the question.** "Safety is mostly marketing", or "alignment is solved, you do RLHF". This suggests you have not thought about it, at a company where colleagues have made career decisions based on exactly these questions.
- **Reciting fears.** Repeating arguments about catastrophic risk without engaging with what can be measured today. This suggests you have adopted a position rather than reasoned towards one.

What works is the approach from module 07's paper discussion: specific, sourced and in proportion. For example:

> "The failure I find most concerning in current systems is reward hacking, because it is already measurable: true quality can fall while the learned reward keeps rising. I would first reach for a KL penalty towards the original model. But that trades directly against capability, and I don't know of a principled way to set its strength."

That answer has a claim, a reason, a mitigation, and an honest statement of where knowledge runs out. Every section of this module supplies material for an answer of that shape.

Recap

- Alignment replaces a loss function nobody can write with a judge fitted to human comparisons. Everything else in the module follows from that substitution.
- The Bradley–Terry model turns comparisons into rewards, and learns only differences between them.
- Sampling blocks the gradient. The policy gradient restores it by moving the derivative onto the log-probability, which turns training into reward-weighted cross-entropy on the model's own samples.
- Baselines, importance ratios and clipping make that estimate affordable rather than correct. Plain REINFORCE was already correct.
- RLHF's best policy is the reference policy reweighted by $e^{r/\beta}$, and DPO follows by cancelling $\log Z$.
- A judge fitted to finite data is wrong somewhere, and hard optimisation finds where: true quality fell 15% while the learned reward kept rising.
- A judge fitted to ordinary inputs says nothing about constructed ones, which is what adversarial examples and prompt injection exploit.
- With different base rates, calibration and equal error rates cannot both hold, so choosing between them is a decision about values rather than a technical detail.
- Memorisation is the exception: it predates the judge, and preference training does not touch it.

The exercise: safety phenomena you can reproduce

What you will build. Seven small experiments, each running in seconds, plus a written part:

1. a Bradley–Terry reward model, and a check that only differences are learned;
2. the closed-form RLHF policy, checked against direct optimisation;
3. DPO, shown to reach the same policy;
4. the reward-hacking curve from section 7;

5. the fairness impossibility, from section 9;
6. a membership-inference attack, and a defence;
7. FGSM and PGD attacks, and adversarial training;
8. written answers to five typical interview questions.

Read this one first. Sections 4 and 5 have no exercise part, because the closed form makes reinforcement learning unnecessary on a three-answer problem. Instead they have a worked script. Run `python policy-gradient.py` from inside the module folder before you start. It reproduces every table in those two sections, with 19 checks, in about 20 seconds, and it is short enough to read end to end.

Time and setup. Allow four to five hours, including the written part. The setup is the same as for module 00.

How to run it. Open `08-ai-safety-alignment/exercise.ipynb`, or run `python exercise.py` from inside the module folder.

How the checks work. They work as in module 00. Before you start, each part reports that it is not implemented:

```
[FAIL] part 1  -- NotImplementedError:

0/1 passed.  Still failing: part 1
```

Step map. This table links each part to the section of the article it practises.

Part	You write	The idea it practises	Section
1	`bradley_terry_loss`, `fit_reward_model`	rewards from preferences	2
2	`optimal_policy`	the RLHF objective	3
3	`dpo_loss`, `train_dpo`	DPO	6
4	`reward_hacking_curve`	reward hacking	7
5	`make_calibrated_population`, `fairness_metrics`, `calibration_error`	the impossibility theorem	9
6	`make_memorisable_data`, `train_classifier`, `membership_inference_auc`	memorisation and privacy	10
7	`make_rings`, `fgsm`, `pgd`, `adversarial_train`	adversarial robustness	8
8	written answers	discussing safety	1, 11

Two details in parts 6 and 7 matter, and the notebook explains both. The membership attack needs random labels and double-precision losses. Without them, it has nothing to detect. And adversarial training must use PGD rather than FGSM, because FGSM-trained models often resist FGSM while failing against PGD.

When you are done. Your reward-hacking curve shows the learned reward rising while the true reward peaks and falls. Your fairness experiment shows unequal error rates despite calibrated scores. Your defences measurably reduce the membership attack's AUC and improve accuracy under PGD. Part 8 should contain a specific, calibrated answer to each question.

If you get stuck.

- For part 2, compare your closed form with the provided numerical optimiser at two values of β.
- For part 4, print both reward columns at each β before checking the shape of the curve.
- For part 5, check calibration first. Everything else depends on the score being calibrated.
- If you are still stuck, `solution.py` has a complete reference for the code parts.

Want more? The notebook ends with four stretch goals. Add a length bias to the reward-hacking experiment and detect it, try the IPO variant of DPO, implement per-example gradient clipping, and build a small automated red-teaming loop. A fifth, if sections 4 and 5 interested you: change `policy-gradient.py` so the baseline is a learned value model rather than the batch mean, and see whether the bias in the table disappears.

Going deeper

Start here

- Amodei et al. (2016), *Concrete Problems in AI Safety*. arXiv:1606.06565. Still the clearest map of practical safety problems, including reward hacking and specification gaming.

Reinforcement learning itself

- Williams (1992), *Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning*, Machine Learning 8. The REINFORCE estimator of section 4, in its original paper.
- Schulman et al. (2017), *Proximal Policy Optimization Algorithms*. arXiv:1707.06347. Short, readable, and the source of the clipped objective in section 5.
- Schulman et al. (2016), *High-Dimensional Continuous Control Using Generalized Advantage Estimation*, ICLR. arXiv:1506.02438. Where the advantage estimates used in practice come from.
- Shao et al. (2024), *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. arXiv:2402.03300. Section 4 introduces GRPO, and explains the group baseline that replaces the value model.

Learning from preferences

- Christiano et al. (2017), *Deep Reinforcement Learning from Human Preferences*, NeurIPS. arXiv:1706.03741. The paper that started learning rewards from human comparisons.
- Ouyang et al. (2022), *Training language models to follow instructions with human feedback*, NeurIPS. arXiv:2203.02155. The InstructGPT paper, which brought RLHF to language models.
- Bai et al. (2022), *Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback*. arXiv:2204.05862.
- Bai et al. (2022), *Constitutional AI: Harmlessness from AI Feedback*. arXiv:2212.08073.
- Rafailov et al. (2023), *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*, NeurIPS. arXiv:2305.18290. Its section 4 is the derivation from section 6 of this article.
- Bradley & Terry (1952), *Rank Analysis of Incomplete Block Designs*, Biometrika 39(3).

Failure modes

- Gao, Schulman & Hilton (2023), *Scaling Laws for Reward Model Overoptimization*, ICML. arXiv:2210.10760. The reward-hacking curve of section 7, measured at scale rather than in miniature.
- Krakovna et al. (2020), *Specification gaming: the flip side of AI ingenuity*, DeepMind. A blog post with a list of dozens of real examples.
- Shah et al. (2022), *Goal Misgeneralisation: Why Correct Specifications Aren't Enough For Correct Goals*. arXiv:2210.01790.
- Sharma et al. (2023), *Towards Understanding Sycophancy in Language Models*. arXiv:2310.13548.
- Hubinger et al. (2019), *Risks from Learned Optimization in Advanced Machine Learning Systems*. arXiv:1906.01820.

Adversarial robustness and privacy

- Goodfellow, Shlens & Szegedy (2015), *Explaining and Harnessing Adversarial Examples*, ICLR. arXiv:1412.6572. The FGSM attack.
- Madry et al. (2018), *Towards Deep Learning Models Resistant to Adversarial Attacks*, ICLR. arXiv:1706.06083. PGD, and adversarial training with it.
- Zou et al. (2023), *Universal and Transferable Adversarial Attacks on Aligned Language Models*. arXiv:2307.15043. The GCG jailbreak attack.
- Greshake et al. (2023), *Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection*. arXiv:2302.12173.
- Carlini et al. (2021), *Extracting Training Data from Large Language Models*, USENIX Security. arXiv:2012.07805.
- Carlini et al. (2022), *Quantifying Memorization Across Neural Language Models*. arXiv:2202.07646.
- Abadi et al. (2016), *Deep Learning with Differential Privacy*, CCS. arXiv:1607.00133. The DP-SGD method of section 10.

Fairness

- Chouldechova (2016), *Fair prediction with disparate impact*. arXiv:1610.07524. The more readable of the two impossibility proofs.
- Kleinberg, Mullainathan & Raghavan (2016), *Inherent Trade-Offs in the Fair Determination of Risk Scores*. arXiv:1609.05807.
- Hardt, Price & Srebro (2016), *Equality of Opportunity in Supervised Learning*, NeurIPS. arXiv:1610.02413.
- Barocas, Hardt & Narayanan (2023), *Fairness and Machine Learning* — https://fairmlbook.org. A free book, and the best full treatment.

Evaluation and governance

- Ganguli et al. (2022), *Red Teaming Language Models to Reduce Harms*. arXiv:2209.07858.
- Perez et al. (2022), *Red Teaming Language Models with Language Models*. arXiv:2202.03286.
- Anthropic, *Responsible Scaling Policy* — https://www.anthropic.com/rsp. One example of tying deployment to capability evaluations. OpenAI's Preparedness Framework and Google DeepMind's Frontier Safety Framework follow a similar approach.
- Templeton et al. (2024), *Scaling Monosemanticity* — https://transformer-circuits.pub/2024/scaling-monosemanticity/. Interpretability with sparse autoencoders, applied to a production model.

Next: module 09 returns to the hardware, and writes attention as a GPU kernel.